

LAB : *Workflows avec langGraph***Table des matières**

PARTIE 1 : Nouveau projet Hello Graph	2
PARTIE 2 : Un Workflow avec deux étapes	3
PARTIE 3 : Ajouter un Reducer	4
PARTIE 4 : Ajouter un état du graphe de type message.....	5
PARTIE 5 : Workflow conditionnel.....	6
PARTIE 6 : Workflow en boucle	7

PARTIE 1 : Nouveau projet Hello Graph

Créer un dossier de TP et ouvrez le dans vscode

Ouvrez un nouveau terminal et vérifiez le version de python et uv

```
PS C:\Users\user\Desktop\langchainPrompt> python --version
Python 3.12.10
PS C:\Users\user\Desktop\langchainPrompt> uv --version
uv 0.6.14 (a4cec56dc 2025-04-09)
```

Créer un nouveau projet avec uv

uv init

uv venv

Activez l'environnement virtuelle

.venv\Scripts\activate

Créer un fichier hello_graph.py

```
from langgraph.graph import StateGraph, MessagesState, START, END
```

```
def hello_node(state: MessagesState):
    # Return a state update: add one assistant message
    return {"messages": [{"role": "ai", "content": "Hello World"}]}
```

```
builder = StateGraph(MessagesState)
builder.add_node("hello", hello_node)
builder.add_edge(START, "hello")
builder.add_edge("hello", END)
```

```
graph = builder.compile()
```

```
result = graph.invoke({"messages": [{"role": "user", "content": "Hi"}]})
print(result["messages"][-1].content)
```

```
# START -> func hello_node() -> END
```

Ajouter au fur et à mesure les dépendances avec uv add nomDeLaDépendance
Exécuter le fichier .py avec : uv run --active python hello_graph.py

PARTIE 2 : Un Workflow avec deux étapes

Créer dans le dossier du projet un nouveau dossier nommé workflows

Créer un fichier two_step_workflow.py

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    topic: str
    joke: str

def refine_topic(state: State):
    return {"topic": state["topic"] + " (and cats)"}

def write_joke(state: State):
    return {"joke": f"Here is a joke about {state['topic']}. "}

builder = StateGraph(State)
builder.add_node("refine_topic", refine_topic)
builder.add_node("write_joke", write_joke)

builder.add_edge(START, "refine_topic")
builder.add_edge("refine_topic", "write_joke")
builder.add_edge("write_joke", END)

graph = builder.compile()

result = graph.invoke({"topic": "ice cream", "joke": ""})

print(result)
```

PARTIE 3 : Ajouter un Reducer

Un reducer est une fonction qui choisit comment fusionner une nouvelle valeur avec l'ancienne valeur dans le state

Dans LangGraph plusieurs nodes modifient le même state, ces modifications arrivent progressivement, parfois en parallèle. Donc il faut une règle pour éviter d'écraser les données et de perdre de l'information

Créer un fichier reducers_demo.py

```
from typing_extensions import TypedDict, Annotated
from operator import add
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    topic: str
    log: Annotated[list[str], add]

def step_a(state: State):
    return {"log": [f"step_a saw topic={state['topic']}"]}

def step_b(state: State):
    return {"log": [f"step_b finishing topic={state['topic']}"]}

def step_c(state: State):
    return {"log": [f"step_c finishing topic at last={state['topic']}"]}

builder = StateGraph(State)
builder.add_node("step_a", step_a)
builder.add_node("step_b", step_b)
builder.add_node("step_c", step_c)

builder.add_edge(START, "step_a")
builder.add_edge("step_a", "step_b")
builder.add_edge("step_b", "step_c")
builder.add_edge("step_c", END)

graph = builder.compile()

result = graph.invoke({"topic": "langgraph", "log": []})

print(result['log'])
```

Ici log est une liste, et si plusieurs valeurs arrivent on utilise add pour les fusionner, add est le reducer

PARTIE 4 : Ajouter un état du graphe de type message

Créer un fichier message_state.py

```
from typing_extensions import TypedDict, Annotated
from operator import add
from langchain_core.messages import BaseMessage
from langgraph.graph import StateGraph, START, END

class ChatState(TypedDict):
    messages: Annotated[list[BaseMessage], add]
    steps: int

def echo_node(state: ChatState):
    return {
        "messages": [{"role": "ai", "content": f"Step {state['steps']}: got your message."}],
        "steps": state["steps"] + 1,
    }

def echo_node_1(state: ChatState):
    return {
        "messages": [{"role": "ai", "content": f"Step {state['steps']}: got your message 1."}],
        "steps": state["steps"] + 1,
    }

builder = StateGraph(ChatState)
builder.add_node("echo", echo_node)
builder.add_node("echo_1", echo_node_1)

builder.add_edge(START, "echo")
builder.add_edge("echo", "echo_1")
builder.add_edge("echo_1", END)

graph = builder.compile()

result = graph.invoke({"messages": [{"role": "user", "content": f"hello"}], "steps": 1})

print(result["messages"])
```

PARTIE 5 : Workflow conditionnel

Ce code construit un workflow avec LangGraph qui génère une blague à partir d'un sujet, puis décide dynamiquement si elle doit être améliorée ou non. La fonction `check_joke` agit comme un routeur conditionnel : si la blague contient déjà un "!", le graphe s'arrête, sinon il passe par le nœud d'amélioration. Au final, le système exécute soit un simple flux `generate` et `end`, soit `generate`, `improve` et `end`, en mettant à jour l'état (`joke`, `improved`) à chaque étape.

Créer un fichier `conditional_workflow.py`

```
from typing_extensions import TypedDict
from typing import Literal
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    topic: str
    joke: str
    improved: str

def generate_joke(state: State):
    return {"joke": f"A joke about {state['topic']} maybe ? "}

def improve_joke(state: State):
    return {"improved": state['joke'] + " !!!"}

def check_joke(state: State) -> Literal["improve", "end"]:
    if "!" in state["joke"]:
        return "end"
    return "improve"

builder = StateGraph(State)
builder.add_node("generate_joke", generate_joke)
builder.add_node("improve_joke", improve_joke)

builder.add_edge(START, "generate_joke")
builder.add_conditional_edges("generate_joke", check_joke, {"improve": "improve_joke", "end": END})
builder.add_edge("improve_joke", END)

graph = builder.compile()

result = graph.invoke({"topic": "cats", "joke": "", "improved": ""})

print(result)
```

PARTIE 6 : Workflow en boucle

Ce code construit une boucle avec LangGraph où le nœud step incrémente la variable n et ajoute une trace dans log à chaque itération. La fonction should_continue agit comme une condition d'arrêt : tant que $n < 5$, le graphe revient sur step (boucle), sinon il s'arrête (END). Au final, le programme exécute plusieurs itérations jusqu'à atteindre 5, affiche l'état final (valeur de n et historique log), puis génère une image du graphe sous forme de fichier PNG.

Créer un fichier workflow_loop.py

```
from typing_extensions import TypedDict
from typing import Literal
from langgraph.graph import StateGraph, START, END
from IPython.display import Image, display

class State(TypedDict):
    n: int
    log: list[str]

def step(state: State):
    n = state["n"] + 1
    return {"n": n, "log": state["log"] + [f"n is now {n}"]}

def should_continue(state: State) -> Literal["again", "stop"]:
    return "again" if state["n"] < 5 else "stop"

builder = StateGraph(State)
builder.add_node("step", step)

builder.add_edge(START, "step")
builder.add_conditional_edges("step", should_continue, {"again": "step",
"stop": END})

# START -> step n + 0 = 1, -> step n + 1 = 2 -> END
graph = builder.compile()

result = graph.invoke({"n": 0, "log": []})

print(result)

png_bytes = graph.get_graph().draw_mermaid_png()

with open("graph2.png", "wb") as f:
    f.write(png_bytes)
```